DDL COMMANDS

create a table department in mysql database with the following schema and constraints:

department (dno, dname, location)

dno (Identifier for department number) is an Integer attribute. dname (Identifier for department name) is a variable character attribute of length 50. location (Identifier for department location) is a variable character attribute of length 50.

Constraints to set for the table department :

1. dno is the key attribute
2. dname should not be empty and must be unique
3. set the default 'location' to "Main Block"

query :

```
CREATE TABLE department(
  dno Int,
  dname varchar(50) not null,
  location varchar(50) default "Main Block",
  Constraint department_dno_pkey primary key(dno),
  CONSTRAINT department_dname_unq UNIQUE(dname)
  );
```

Create a table employee in MySQL database with the following schema and Constraints:

employee (empno, name, gender, salary, email, dno)

empno (Identifier for employee number) is an Integer attribute. name (Identifier for employee name) is a Variable character attribute of length 50. gender is a fixed length of attribute

with a single character. Salary is a floating point attribute with 5 decimal points and a 2 fractional points. Email is a variable character attribute of length 100. dno (identifier for department number) is an Integer attribute.

constraints to set for the table employee:

1. empno is the key attribute
2. name should not be empty
3. check that the salary is greater than 0
4. email must be unique.

query:

```
CREATE TABLE employee(
    empno int,
    name Varchar(50) NOT NULL,
    gender char,
    Salary decimal(7,2),
    email Varchar(100),
    dno int,
    CONSTRAINT employee_empno_pkey PRIMARY KEY(empno),
    CONSTRAINT employee_salary_chk CHECK(salary>0),
    CONSTRAINT employee_email_unq UNIQUE(email)
    );
```

1. Add a column phone to employee

```
ALTER TABLE employee ADD phone char(10);
```

2. change the size of name to 100 characters.

```
ALTER TABLE employee MODIFY COLUMN name Varchar(100);
```

3. Add unique constraint to phone

```
ALTER TABLE employee
ADD CONSTRAINT employee_phone_unq UNIQUE(phone);
```

4. Add check constraint to salary must be >= 10000

```
ALTER TABLE employee
ADD CONSTRAINT employee_salary1_chk CHECK (salary >= 10000);
```

5. Drop the email column

```
ALTER TABLE employee DROP COLUMN email;
```

6. Add Composite unique constraint on (name, phone)

```
ALTER TABLE employee
ADD CONSTRAINT employee_namephone_unq UNIQUE(name, phone);
```

7. Rename column name to ename

```
ALTER TABLE employee RENAME COLUMN name TO ename;
```

8. Rename table department to dept

```
ALTER TABLE department RENAME TO dept;
```

DML COMMANDS

Based on the Following database schema:

department(dno, dname, location)

employee(empno, name, gender, salary, email, dno)

1. Insert the Following records to the table department:
   - ↳ 10, HR, Block A
   - ↳ 20, Sales, Block B
   - ↳ 30, IT, Block C
   - ↳ 40, Administration, Block D
   - ↳ 50, Finance, Block E

2. Insert the Following records to the table employee:
   - 1, Alice, F, 30000, alice@gmail.com, 10
   - 2, Bob, M, 40000, bob@gmail.com, 20
   - 3, charlie, M, 45000, charlie@gmail.com, 30
   - 4, David, M, 35000, david@gmail.com, 40
   - 5, Suzanne, 50000, suzanne@gmail.com, 50
   - 6, Talha Ahmed, 'M', 25000, 'talha@gmail.com, 10
   - 7, Ayat Nashwa, 'F', 27000, 'ayat@gmail.com, 10
   - 8, Mehek', 'F, 32000, mehak@gmail.com, 20
   - 9, Zara, F, 38000, zara@gmail.com, 30
   - 10, Dawood, M, 42000, dawood@gmail.com, 40
   - 11, Ayzal Mariyam, F, 29000, ayzal@gmail.com, 40

1. query

```
INSERT INTO department VALUES (10, 'HR', 'Block A'), (20, 'Sales',
'Block B'), (30, 'IT', 'Block C'), (40, 'Administration', 'Block D')
(50, 'Finance', 'Block E');
```

2. query

```
INSERT INTO employee VALUES (1, 'Alice', 'F', 30000, 'alice@gmail.com', 10), (2, 'Bob', 'M', 40000, 'bob@gmail.com', 20), (3, 'Charlie', 'M', 45000, 'charlie@gmail.com' 30), (4, 'David', 'M', 35000, 'david@gmail.com', 40);
INSERT INTO employee (empno, name, salary, email, dno) VALUES (5, 'Suzanne', 50000, 'Suzanne@gmail.com', 50);
INSERT INTO employee VALUES (6, 'Talha Ahmed', 'M', 25000 'talha@gmail.com', 10), (7, 'Ayat Nashwa', 'F', 27000, 'ayat @gmail.com', 10), (8, 'Mehak', 'F', 32000, 'mehak@gmail.com' 20), (9, 'Zara', 'F', 38000, 'zara@gmail.com', 30), (10, Dawood 'M', 42000, 'dawood@gmail.com', 40), (11, 'Ayzal Mariyum', 'F', 2900; 'ayzal@gmail.com', 40);
```

3. perform the Following operations.

↳ Increase the salary of all employees by 10%

```
UPDATE employee SET salary = salary * 1.10;
```

↳ Update the location of department 'Sales' to Block G

```
UPDATE department SET location = 'Block G' WHERE dname = 'Sales';
```

↳ Delete all employees whose salary is less than 35000

```
DELETE FROM emplee WHERE Salary < 35000;
```

↳ Update salary by adding 5000 For employees working in department number 30 and drawing a salary less than 50000

```
UPDATE employee SET Salary = Salary + 5000
  WHERE dno = 3 and Salary < 50000;
```

SQL QUERY COMMANDS.

Based on the following database schema:

department (dno, dname, location)

employee (empno, name, gender, salary, email, dno)

perform the following queries:

1. Display all details of all employees

```
SELECT * FROM employee;
```

2. Display only the employee name and salary

```
SELECT name, salary FROM employee;
```

3. List all employees who belong to department number 10 and draw salary between 20,000 and 30,000

```
SELECT * FROM employee
WHERE dno = 10 AND SALARY BETWEEN 20000 AND 30000;
```

4. Display employees whose name starts with the letter 'A'.

```
SELECT * FROM employee WHERE name LIKE "a%";
```

5. Display employees who work in the department named 'IT'.

```
SELECT * FROM employee NATURAL JOIN department
WHERE dname = "IT";
```

6. List employees whose salary is greater than the average salary of all employees

```
SELECT * FROM employee
WHERE Salary > (SELECT AVG(salary) FROM employee);
```

7. Display the number of employees in each department

```
SELECT dno, count(*) as employee_count FROM employee
GROUP BY dno;
```

8. List departments having more than 3 employees.

```
SELECT dno, count(*) as employee_count
FROM employee GROUP BY dno HAVING COUNT(*) > 3;
```

9. List departments where the average salary is greater than 30000.

```
SELECT dno, AVG(salary) as avg_salary
FROM employee GROUP BY dno
HAVING AVG(salary) > 30000;
```

10. Display the department name where the highest paid employee work.

```
SELECT dname FROM employee NATURAL JOIN department
WHERE salary = (SELECT MAX(salary) FROM employee);
```